

CM Todos a Todos y DAG's

Ezequiel Companeetz, Rafael Romani, Tobias Carreira Munich

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

2025

Programa de hoy (DAG's)

- 1 Aclaraciones importantes
- 2 Repaso de definiciones y algoritmos
- 3 Aplicaciones de CM y DAG
- 4 Ejercicios introductorios
- 5 Ejercicios CM todos a todos
- 6 Conclusiones

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

Problemas de CM

Problemas a modelar

- Camino mínimo entre dos vértices
- Camino mínimo entre un vértice y el resto de vértices
- Camino mínimo entre todos los pares de vértices

¿Qué algoritmos resuelven estos problemas?

Variantes de cada problema

- Grafos con aristas no dirigidas
- Digrafo sin pesos en las aristas
- Digrafo con/sin aristas de peso negativo
- Digrafo con/sin ciclos de peso negativo.

Algoritmos de CM

Variables a tener en cuentas

- Cada algoritmo se puede medir de muchas maneras distintas:
 - Complejidad temporal
 - Complejidad espacial
 - Implementación sencilla o compleja
 - ¿Calcula información parcial útil? ¿Cuál es su invariante? (i.e. Qué pasa si modifico el grafo luego de haber resuelto el problema?)

Algoritmos de la clase de hoy

- Dijkstra (uno contra todos)
- Floyd-Warshall (todos contra todos)
- Dantzig (todos contra todos)

Repaso de propiedades

Principio de optimalidad

Todo sub camino de un camino mínimo es un camino mínimo, siempre que no haya ciclos de pesos negativos.

Árbol de caminos mínimos

Sea G un grafo, s un vértice de G y G no posee aristas de costo negativo alcanzables desde s . Sea W el conjunto de vértices alcanzables desde s en G . Entonces existe un árbol orientado T con raíz s y $V(T) = W$, tal que para todo $v \in W$, el camino de s a v en T es un camino mínimo de s a v en G .

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

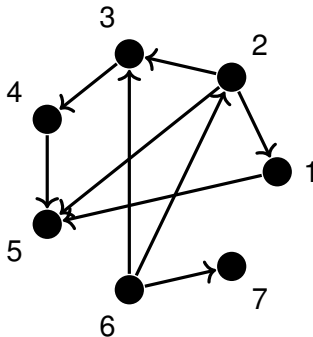
3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

DAGs

Definición

Un digrafo D es un DAG (*directed acyclic graph*) si no tiene ciclos dirigidos.



¿Cómo puedo saber si un grafo es un DAG? DFS o BFS, en $O(n + m)$.

DAGs y orden topológico

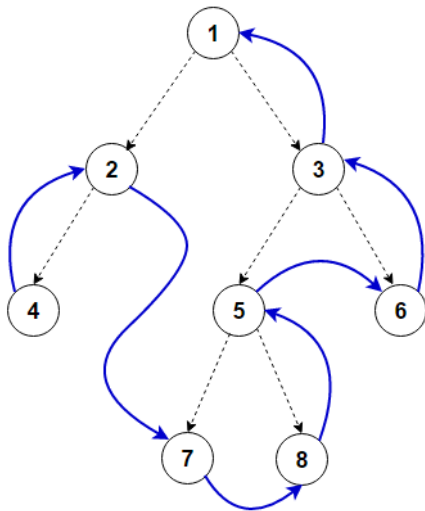
Orden Topológico

Definimos al orden topológico (o también conocido como *orden de actualización seguro inverso*) de un digrafo $D = (V, E)$ como un ordenamiento de los nodos $v_1 v_2 \dots v_n$ tal que para todo eje $v_i v_j \in E$ vale que $i < j$.

¿Cuál sería un orden topológico para nuestro DAG anterior? ¿Este es válido $v_6, v_7, v_2, v_3, v_4, v_5, v_1$? ¿Y este $v_6, v_7, v_2, v_3, v_4, v_1, v_5$?

- ¿Todo digrafo tiene un orden topológico?
- ¿Todo digrafo que admita un orden topológico es un DAG?
- ¿Todo DAG tiene un orden topológico? ¿Es único ese orden topológico?
- ¿Cuál es la máxima cantidad de ejes que puede tener un DAG de n nodos?

Orden Topológico (posible) = Post order invertido



Postorder: 4, 2, 7, 8, 5, 6, 3, 1

Algoritmo de Kahn

- ➊ Inicializar una lista vacía L que contendrá los nodos en orden topológico.
- ➋ Inicializar un conjunto S con todos los nodos que no tengan aristas de entrada (grado de entrada = 0).
- ➌ Mientras S no esté vacío, repetir los siguientes pasos:
 - ➊ Eliminar un nodo n de S .
 - ➋ Añadir n a la lista L .
 - ➌ Para cada nodo m con una arista (n, m) de n a m :
 - ➊ . Eliminar la arista (n, m) del grafo.
 - ➋ . Si m no tiene más aristas de entrada, añadir m a S .
- ➍ Si el grafo aún tiene aristas, entonces tiene al menos un ciclo, y no es posible un orden topológico.
- ➎ De lo contrario, L contiene un orden topológico de los nodos.

Problemas sobre DAGs

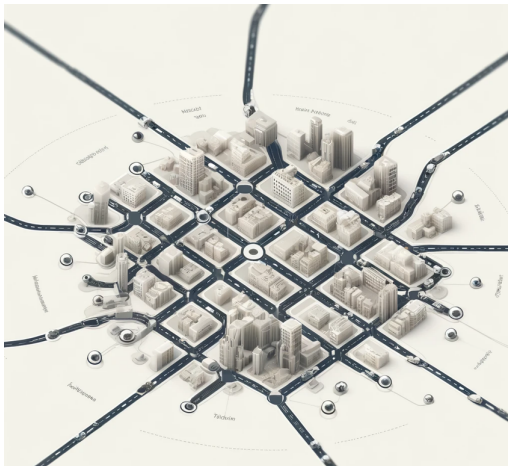
Sobre DAGs muchos problemas son fáciles, gracias al uso del orden topológico. Este lo podemos obtener con el algoritmo de Kahn o invirtiendo el post-order de *DFS*. Por ejemplo:

Problemas

- ¿Cuál es la cantidad de caminos que van de v a w ?
- Si el digrafo es pesado, ¿Cuál es el camino de menor costo de v a w ?
- ¿Y el de mayor peso?
- ¿Con qué técnica se resuelven estos problemas? ¿Qué propiedad de los caminos mínimos utilizamos?

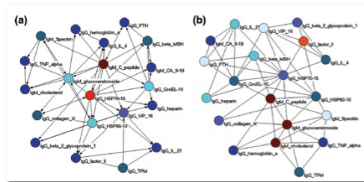
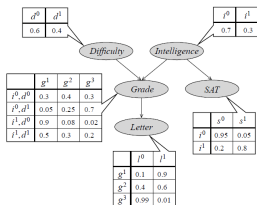
¿Por qué no podemos hacer lo mismo en digrafos generales?

Aplicaciones de CM todos a todos



Manejo de tráfico urbano, buscar recorridos óptimos entre varios puntos

Problemas que podemos modelar cómo DAGs



Redes Bayesianas. Son DAG's que modelan causalidad.

Redes de dependencias

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

Sasha is back

Enunciado

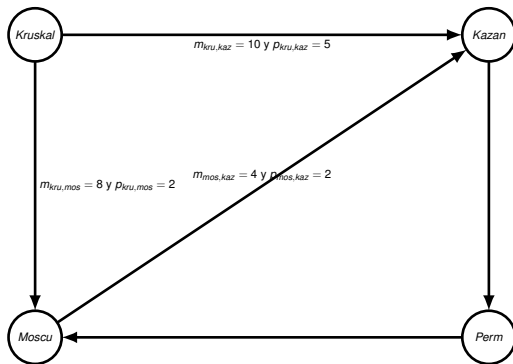
Sasha termino de cursar un cuatri agotador, por lo que va a volver a *Kruskal* para irse de vacaciones. Ya terminaron las obras con el mapa anterior que le dejo al presidente, e inclusive se agregaron más rutas. Así quedó un mapa de n ciudades y m rutas. Cada ruta e_i tiene un cierto peaje asociado p_i y se tarda m_i minutos en recorrerla. Queremos encontrar el camino de *Kruskal* a *Kazan* que minimice el costo de los peajes a pagar y que se pueda recorrer en menos de t^a minutos.

^a t es "chico" ($t \in \mathcal{O}(n)$)

Especificaciones

- El formato del input es un número N y M siendo las ciudades y rutas respectivamente. Seguidos de M líneas de la forma c_x, c_y, p_i, m_i con las dos ciudades que conecta cada ruta y su respectivo tiempo y peaje.

Dibujemos un ejemplo



- ¿Quiénes serían V y E en este caso?
- ¿Hay algún problema con que tenga ciclos este grafo?
- ¿Qué camino elegiría si $t = 11$ ¿Con $t = 13$?

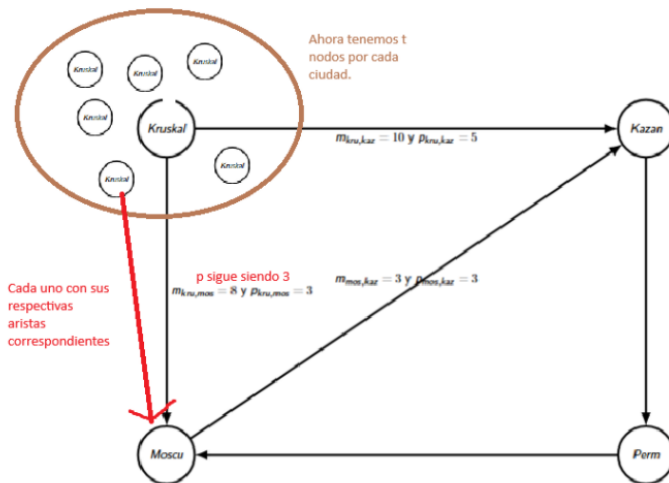
Analicemos el problema

- Son “casi” dos caminos mínimos en simultáneo: para el tiempo no queremos un camino mínimo, e incluso puede ser contraproducente.
- Recordemos que t es “chico” ¿Cómo se les ocurre que podemos modelar este problema?
- Podemos modificar nuestros algoritmos de camino mínimo para que utilicen una nueva función de peso! Así siempre los podemos adaptar al problema que necesitemos!
- Acá tenemos algunos ejemplos de nuevas funciones de peso ¿Cuál es la correcta?
 - $w(x, y) = p_{x,y} + m_{x,y}$
 - $w(x, y) = p_{x,y} * m_{x,y}$
 - $w(x, y) = p_{x,y} * (m_{x,y} < t)$
- ¡Ninguna es correcta!

Una idea mejor

- La idea anterior vimos que tenía varias complicaciones. ¿Qué otra opción se les ocurre?
- ¡Podemos llevar la cuenta del tiempo en los nodos! Para esto podemos crear un digrafo G_t ¿Cómo nos quedarían los vértices de G_t ?
- Sus vértices son los pares (c, t') donde $c \in V$ y $0 \leq t' \leq t$. ¿Qué aristas los conectan?
- Conectamos (c_1, t_1) con (c_2, t_2) con un eje de peso p_j si el eje $c_1 c_2 = e_j$ cumple $m_j + t_1 = t_2$. ¿Qué representa cada arista?

G_t de bajo presupuesto



Solución

- ¿Cómo hacemos para resolver el problema a partir de este digrafo G_t ? ¿Ya está completo nuestro digrafo?
- A todos los nodos de la forma $(Kazan, t')$ para todos los t' le agregamos un eje de costo 0 al nodo *final* ¿Cómo resolvemos el problema?
- Queremos encontrar un camino de costo mínimo de $(Kruskal, 0)$ a *final* en G_t ¿Cuál es el tamaño de G_t ? ?
- Tiene $\mathcal{O}(nt)$ nodos y $\mathcal{O}(mt)$ ejes. Ahora podemos utilizar Dijkstra para calcular el camino en $\mathcal{O}(mt \log(nt))$. ¿Se puede hacer en mejor tiempo? ¿Qué forma tiene el digrafo?

DAGs, here we go again



- Es un digrafo sin ciclos. ¿Por qué sabemos que no tiene ciclos?
¿Qué implicaría un ciclo en nuestro modelo?
- ¿Cómo podemos calcular el camino mínimo en un DAG?

CM de v a s en DAGs

$$d_s(v) = \begin{cases} 0, & \text{si } v = s \\ \min\{w(u, v) + d_s(u) : u \rightarrow v \in E\}, & \text{si no.} \end{cases}$$

s es el nodo desde el cual estamos calculando las distancias.

- ¿Vale esto siempre, aunque no tenga ciclos?
- ¿Me sirve para hacer programación dinámica?
- ¿Cómo me queda la complejidad del algoritmo ahora?

CM de v a s en DAGs

$$d_s(v) = \begin{cases} 0, & \text{si } v = s \\ \min\{w(u, v) + d_s(u) : u \rightarrow v \in E\}, & \text{si no.} \end{cases}$$

- ¿Vale esto siempre, aunque no tenga ciclos? *Sí, es matemáticamente correcto*
- ¿Me sirve para hacer programación dinámica? *Solo si no tengo ciclos. ¿Por qué?*
- ¿Cómo me queda la complejidad del algoritmo ahora? *Lineal en base al tamaño del grafo, en este caso es $\mathcal{O}(mt + nt)$*
- Con esto podríamos escribir la función top down. Si queremos la forma bottom-up deberíamos calcular el orden topológico y comenzar en s hasta llegar a t para obtener el mínimo.

Solución final

- Construyo el grafo G_t , agregando al nodo *final*. $\mathcal{O}(nt + mt)$
- Calculo camino mínimo de (*Kruskal*, 0) a *final*. $\mathcal{O}(nt + mt)$
- Le damos el camino a Sasha así se puede ir de vacaciones. $\mathcal{O}(1)$
- Complejidad total: $\mathcal{O}(nt + mt)$.
- Si el valor de "t" no está acotado, entonces nuestra solución puede ser exponencial.

Variaciones Ejercicio 1

Posibles variaciones del ejercicio

¿Cómo cambiaría nuestra solución si ahora nos dicen que queremos encontrar la ruta que cumpla las mismas especificaciones (de tiempo $< t$ y menor peaje) pero para todas las ciudades?

- Deberíamos agregar un nodo *final* para cada una de las ciudades.
- Luego, con los algoritmos que vimos no solo tenemos el camino mínimo de un vértice a otro, sino que podemos obtener el camino mínimo de *Kruskal* a todos. ¿Cómo?
- Utilizando Dijkstra o PD sobre DAG's, lo cual también nos va a dar el árbol de caminos mínimos.

Variaciones Ejercicio 1

Posibles variaciones del ejercicio

- ¿Qué ocurre si ahora queremos contar todos los recorridos posibles desde *Kruskal* a *Kazan* que tarden menos de t minutos?
 - Usando PD cómo tenemos un DAG podemos contar la cantidad de caminos en un tiempo polinomial. Podemos crear una función CantidadRecorridos = CR, que nos calcule la CR de un nodo v a *Kazan*:

$$CR(v = (ciudad, t)) = \begin{cases} \sum_{x \in N^+(v)} CR(x), & \text{si } ciudad \neq \text{Kazan} \\ 1, & \text{if } ciudad = \text{Kazan} \end{cases}$$

- Para pensar: ¿Qué ocurre si ahora Natasha (hermana de Sasha) quiere encontrar el camino que tarde menos tiempo entre todos los que minimizan el costo de los peajes a pagar?^a

^aSpoiler: No se pica tanto

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

Rayuela Rectangular

Enunciado (Parte 1)

Sasha volvió a Argentina y decidió hacer turismo en el parque centenario. Ahí se encontró una rayuela rectangular recién pintada. Esta rayuela, de dimensiones $p \times q$, tiene números todos distintos escritos en los casilleros. Los números no son consecutivos (es decir, la rayuela puede tener el 1 y el 4 pero no el 2) y tampoco hay un orden claro para recorrerla.

Junto a la rayuela Sasha descubrió las reglas particulares para jugar sobre este tablero rectangular:

- Estando en una posición (i, j) solo se puede saltar a aquellas posiciones que están en la misma línea horizontal o vertical que (i, j) . Aparte la casilla destino no pueden estar a mayor distancia que k , y su número debe ser mayor al de la casilla (i, j) .
- Al saltar de un casillero con número x a otro con número $x + r$ se ganan r puntos.
- El juego termina cuando le jugadore no puede realizar más saltos válidos.

Rayuela Rectangular

Enunciado (Parte 2)

Sasha es competitiva, y por ende quiere encontrar el casillero inicial desde el cual puede obtener la máxima cantidad de puntos.

El input de nuestro problema es la matriz R con los valores de cada casilla de la rayuela y la distancia k .

Modelando ando

- Igual que en el caso anterior vamos tener que crear un digrafo $D = (V, E)$ que nos modele cómo sería un recorrido de Sasha para esta rayuela. ¿Cómo se les ocurre modelar estos estados?
- Uno podría ver la similitud con el ejercicio 8 de la práctica 4, el problema es que acá no sabemos si hay una cota para los números. Por lo que definir cada vértice v cómo la tripla $(suma_acumulada, i, j)$ nos podría generar un digrafo inecesariamente grande. ¿Se les ocurre una idea mejor?

Armando nuestro modelo

- Exacto! Armamos un grafo cuyos vértices $v \in V$ son los pares (i, j) donde $0 \leq i < p$ y $0 \leq j < q$. ¿Que aristas vamos a tener?
- Conectamos $v = (i_1, j_1)$ con $w = (i_2, j_2)$ con un eje de peso $R[w]$ si se cumple que:
 - v y w están en la misma línea horizontal o vertical y su distancia es menor k : $w = (i_1 + k_1, j_1) \vee (i_1, j_1 + k_1)$ con $-k \leq k_1 \leq k$.
 - El valor del casillero de w es mayor que el de v : $R[v] < R[w]$.
- ¿Qué representa cada arista? ¿Nos falta representar algún otro estado?

Armando nuestro modelo

- Nos falta modelar que Sasha pueda comenzar en cualquier posición y determinar cuándo es que termina. ¿Que se les ocurre?
- Podemos crear un nodo *inicio* y otro *fin* que representen estos estados. ¿Que aristas van a tener estos?
- *inicio* se va a conectar a todos los vértices de D con costo 0 y todos los vértices $v \in V$ tales que $d_{out}(v) = 0$ (¿Por qué puede pasar esto?) van a tener una arista hacia *fin*.
- Ahora ya tenemos nuestro modelo. ¿Cómo se resuelve este problema? ¿Cumple alguna propiedad nuestro digrafo?

¡Es un DAG!



Representación verídica de Sasha si la existencia de un ciclo fuera probada

Solución

- Armamos nuestro digrafo D con sus nodos y aristas pertinentes. Lo cual nos va a costar $\mathcal{O}(p \cdot q \cdot k)$ en el peor caso.
- Luego calculamos el camino máximo entre *inicio* y *fin* utilizando el algoritmo de camino máximo en DAG's. $\mathcal{O}(p \cdot q \cdot k)$, pues es lineal en base al tamaño del grafo. ¿Sobre qué nodo vamos a correr nuestra función de camino máximo?
- Por último devolvemos el segundo nodo de algún camino máximo, el cual va a ser un casillero inicial desde el cuál puede obtener la máxima cantidad de puntos. $\mathcal{O}(1)$
- ¿Se puede mejorar el modelo? Sí, eso queda para pensar.

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

Tips para DAG's

- Leer bien el enunciado y anotar todos los datos que se nos presentan para poder modelar el problema
- Intentar no modificar los algoritmos y usarlos como caja negra. Esto podemos hacerlo al aprovechar los resultados que te dan los mismos, para así resolver el problema (cómo la matriz de *Floyd – Warshall*). También podemos modelar el problema de tal forma que solo tengamos que correr nuestro algoritmo sobre nuestro modelo y eso nos dé la respuesta (cómo el caso del nodo *final* en el ejercicio 1).
- Estén atentos a si el digrafo con el que están tratando es un *DAG*, puesto que eso puede reducir la complejidad de su algoritmo significativamente.

¿Sale un break?



1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

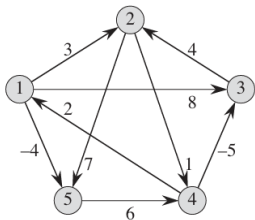
- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

APSP (All-Pairs Shortest Paths)

Objetivo

En esta parte de la clase veremos problemas relacionados a encontrar la longitud de todos los caminos mínimos.

¿Cuál es la complejidad mínima que deberíamos esperar? $O(n^2)$, ya que es el tamaño del output.



$$\begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

Algoritmos para APSP

- Floyd - Warshall: $O(n^3)$
- Dantzig: $O(n^3)$
- Correr n veces camino mínimo uno a todos
 - Con Dijkstra: $O(n \times m \log n)$
 - Con Bellman-Ford: $O(n \times nm)$
- Johnson: $O(nm + nm \log n) = O(nm \log n)$

¿Cuáles son sus invariantes?

Dato interesante

¿Se puede calcular APSP en $O(n^{3-\varepsilon})$?

Subcubic Equivalences Between Path, Matrix, and Triangle Problems*

Virginia Vassilevska Williams[†]

Ryan Williams[‡]

1

[†]<https://people.csail.mit.edu/rrw/tria-mmult.pdf>

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

Optimizando Canciones²

Enunciado

Rasta, que venció el bajón luego de comer tantos alfajores, está aprendiendo a tocar la guitarra. Como le gusta irse de campamento con amigos mochileros, se está esforzando en mejorar su velocidad para tocar acordes para los fogones.

Existen k acordes en total, y la canción con la que está practicando tiene una sucesión de n acordes. A Rasta le toman S_{ij} segundos pasar del acorde i al acorde j , pero a veces le es más rápido cambiar de acorde con arreglos intermedios (es decir tocar otros acordes entre el i y el j).

Se asume que Rasta empieza con la mano acomodada en el primer acorde. ¿Cuál es el mínimo tiempo que le toma tocar la canción?

²Prueben codearlo y ponerlo acá: <https://codeforces.com/gym/102646/problem/C>

Optimizando Canciones

- Por ejemplo, si la canción va del acorde 6 al 7, capaz Rasta hace más rápido yendo del 6 al 3, y luego del 3 al 7.
- Definamos la canción $C = c_1, c_2, \dots, c_n$
- Dado que Rasta puede hacer más rápido pasando por acordes intermedios, es posible que pueda tocar la canción C en menos de $\sum_{i=1}^{n-1} S_{c_i c_{i+1}}$ tiempo.

Optimizando Canciones

- Podemos imaginar un grafo $G = (V, E)$ de k vértices, con una arista $i \rightarrow j$ de peso S_{ij} .
- Buscamos saber, para todo par de acordes i, j , el mínimo tiempo que necesita Rasta para pasar de i a j .
- Esto es nada menos que la matriz de distancias de G .

Optimizando Canciones

- Ahora tenemos que usar algún algoritmo para computar distancias todos con todos.
- ¿Floyd-Warshall o Johnson?
- Para decidir cuál, ¿nuestro grafo es denso o ralo? como hay un $S_{ij} \forall i \neq j$, es completo, así que $|E| \in \Theta(|V|^2)$.
- Usando FW, el valor buscado es $t = \sum_{i=1}^{n-1} \delta(c_i, c_{i+1})$.
- Complejidad: $O(k^3 + n)$, donde k^3 cuesta calcular la matriz con DP y n para recorrer la canción y sumar.

Optimizando Canciones - Modificado

Enunciado

Después de un rato de practicar, Rasta se dio cuenta que la canción todavía no estaba muy fluída y decidió pedir ayuda a sus amigos.

Tuki y Sasha, que también iban al campamento, le sugirieron que podría cambiar un acorde para disminuir el tiempo que le toma tocar la canción. ¿Qué cambio podría minimizar el tiempo total?

Optimizando Canciones - Modificado

- Una manera de responder esta pregunta es yendo acorde por acorde y probando cambiarlo por cada uno de los otros $k - 1$.

- En particular, si cambié el i -ésimo acorde por e , tengo que

$$t_{e \rightarrow c_i} = t \underbrace{-\delta(c_{i-1}, c_i) + \delta(c_{i-1}, e)}_{\text{si } i > 1} \underbrace{-\delta(c_i, c_{i+1}) + \delta(e, c_{i+1})}_{\text{si } i < n}$$

- Casos borde: hay que definir por separado los casos que $i = 1$ o $i = n$ dado que no existen c_0 ni c_{n+1}

Optimizando Canciones - Modificado

Recordando:

- Desigualdad triangular: siempre que $\delta(\cdot, \cdot) \geq 0$, vale que
$$\forall s, p, t \quad \delta(s, t) \leq \delta(s, p) + \delta(p, t)$$

Gracias a la desigualdad triangular no hace falta analizar los $k - 1$ candidatos para cada acorde!

El mínimo se realiza repitiendo un acorde.³

³notar que podría realizarse también sin repetir

Optimizando Canciones - Modificado

Idea de la demostración

Sea $C = c_1, \dots, c_i, c_{i+1}, c_{i+2}, \dots, c_n$ una canción a la que le cambiamos el acorde c_{i+1} por c_{i+1}^* .

$$t^* = \delta(c_1, c_2) + \dots + \delta(c_i, c_{i+1}^*) + \delta(c_{i+1}^*, c_{i+2}) + \dots + \delta(c_{n-1}, c_n)$$

$$t' = \delta(c_1, c_2) + \dots + \delta(c_i, c_i) + \delta(c_i, c_{i+2}) + \dots + \delta(c_{n-1}, c_n)$$

Luego $t^* - t' = \delta(c_i, c_{i+1}^*) + \delta(c_{i+1}^*, c_{i+2}) - \delta(c_i, c_i) - \delta(c_i, c_{i+2}) = \delta(c_i, c_{i+1}^*) + \delta(c_{i+1}^*, c_{i+2}) - \delta(c_i, c_{i+2}) \geq 0$ por desigualdad triangular. Por lo tanto, cambiar el $i + 1$ -ésimo por el c_i era mejor (no peor) que cambiarlo por c_{i+1}^*

Optimizando Canciones - Modificado

Algoritmo

- 1 Calculamos la matriz de distancias y el valor t original. $O(k^3 + n)$.
- 2 Para cada posición $i > 1$, calculamos el $t_{c_{i-1} \rightarrow c_i}$ (el total de cambiar c_i por repetir c_{i-1} , y nos quedamos con el mínimo). $O(n)$
- 3 Costo total: $O(k^3 + n)$.

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

Más trenes

Enunciado

Tuki quiere recorrer su país, y observa que, a diferencia de cuando era joven, ahora hay más estaciones de trenes, lo cual le permite llegar en menos tiempo a los distintos pueblos. Además, Tuki siempre estuvo muy atento a cuando se cambiaban los trenes por otros con distintas velocidades.

Él conoce el mapa de la red de trenes de la Argentina de su infancia, y tiene una línea de tiempo indicando qué nuevas estaciones fueron creadas desde entonces (junto a sus vías) y las veces que se cambiaron las locomotoras (junto con las velocidades medias de las mismas). Tuki está interesado en saber cuánto tardaba en moverse entre ciertos pueblos de acuerdo al desarrollo de las vías hasta el momento y los trenes que iban por ellas.

Más trenes: entrada

Entrada

La entrada consta de una secuencia de *queries* de 3 formas.

- Agregar localidad l , con un lista de rutas r_l .
- Cambiar tren t , donde t indica la velocidad media del tren nuevo.
- ¿Cuánto se tarda en ir de la ciudad i a la j ?

Más trenes: solución

- Tenemos que proponer una estructura de datos para ir procesando las queries y guardando información. Vayamos una por una.
- ¿Qué podemos hacer cuando recibimos `Agregar localidad l`? Tenemos dos estrategias:
 - E1 Agregar el nodo al grafo con sus ejes ¿Complejidad? $O(n)$.
 - E2 Agregar la localidad y **recalcular** las distancias mínimas, a lo Dantzig ¿Complejidad? $O(n^2)$.
- Cambiar tren `a`.
 - E1 Guardar la velocidad del tren actual.
 - E2 Ídem.

Más trenes: solución

- ¿Cuánto tardo de i a j ?
 - E1** Hay que calcular la distancia ¿Complejidad? $O(m \log n)$.
 - E2** Ya tenemos calculado todo ¿Complejidad? $O(1)$.
- ¿Qué solución es mejor? Depende de las queries.

Más trenes: solución

- Sean L las queries de agregar localidad, T las de actualizar tren y Q las de consulta. Las complejidades de cada estrategia son
 - E1** $O(Ln + T + Qm \log n) \sim O(n^2 + T + Qm \log n)$.
 - E2** $O(Ln^2 + T + Q) \sim O(n^3 + T + Q)$.
- ¿Cuándo es mejor **E1**? Cuando Q es chico. Caso contrario, es mejor **E2**.

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

String problem⁴

Enunciado

Sea Σ un conjunto de símbolos, y sean $a, b \in \Sigma^*$ dos cadenas de símbolos de Σ . Para cada par de símbolos s_1, s_2 sabemos el costo $w_{s_1 s_2}$ de reemplazar s_1 por s_2 (el cual puede ser infinito, indicando que no se puede hacer ese cambio **directo**).

En cada paso podemos tomar un carácter s_1 de a y cambiarlo por otro s_2 pagando el costo asociado w_{s_1, s_2} . Nos piden decidir si es posible obtener b a partir de a , y el menor costo que se debe pagar en caso de que sea posible.

⁴<https://codeforces.com/contest/33/problem/B>

String problem: solución

- Claramente, si $|a| \neq |b|$ entonces la respuesta es que no se puede obtener b a partir de a .
- ¿Podremos descomponer el problema? Se puede pensar a cada posición i de la cadena a como un problema distinto, ya que las posiciones quedan fijas.
- Fijemos un i ¿Cómo encontramos la forma más barata de ir de a_i a b_i ?
- Podemos armar un grafo G de $|\Sigma|$ nodos y $O(|\Sigma|^2)$ ejes donde el eje de s_1 a s_2 indica el costo asociado a reemplazar s_1 por s_2 . Luego, la distancia mínima en G de a_i a b_i nos dice el costo mínimo a pagar.

String problem: solución

- Podemos precalcular todas las distancias con Floyd ¿Qué complejidad final queda? $O(|\Sigma|^3 + |a|)$.
- ¿Y si no precalculamos? Podemos usar Dijkstra en cada paso, lo cual nos da una complejidad $O(|a| m^2 \log |\Sigma|)$. Dónde m es la cantidad de pares de letras que se pueden transformar directamente entre si.
- ¿Cuál es mejor? Depende de $|a|$ y de m : si $|a|$ es grande o m es chico entonces conviene precalcular, sino no.

1 DAG's: Definiciones y Algoritmos

- Camino mínimo
- DAGs

2 Aplicaciones de CM y DAG's

- Ejercicio 1
- Ejercicio 2
- Tips

3 APSP – All-Pairs Shortest Path

- Camino mínimo
- Optimizando canciones
- Más trenes
- String problem
- Manic moving

Manic Moving⁵

Enunciado

Homero trabaja para MP (Muchos Productos) y saliendo al trabajo piensa en los locales que tiene que visitar hoy con su camión. Conoce el grafo G de la ciudad, y los locales $l_1, \dots, l_k$ que debe visitar, **en ese orden**.

Como los paquetes son grandes, solo puede llevar dos a la vez (es decir, tiene que volver a central de vez en cuando para cargar nuevos paquetes).

Sabiendo lo que Homero tarda en moverse entre cada par de nodos de la ciudad, debemos indicar cuál es el menor tiempo que le toma repartir todos los productos.

⁵<https://codeforces.com/gym/101223/attachments>

Manic Moving: solución

- Supongamos que Homero solo puede llevar un paquete a la vez ¿Cuál es la solución? Tiene que concatenar caminos mínimos de la central c a l_1 , luego de vuelta a la central, luego a l_2 , etc.
- Ahora supongamos que tiene espacio extra, y va a l_1 con el segundo paquete ¿Qué opciones tiene? Puede ir directo a l_2 , o pasar por central para cargar el paquete de l_3 .
- Uno podría argumentar que si sale con dos paquetes entonces visita dos lugares seguidos y después vuelve.
- Luego en cada paso hay dos opciones:
 - Agarra un paquete y va y vuelve al siguiente local.
 - Agarra dos paquetes, visita los dos locales y luego vuelve.

Manic Moving: solución

Podemos escribir una fórmula recursiva que resuelva el problema

$$hom(i) = \begin{cases} \infty & \text{si } i > k + 1 \\ 0 & \text{si } i = k + 1 \\ \min \begin{cases} hom(i + 1) + \delta(c, l_i) + \delta(l_i, c) \\ hom(i + 2) + \delta(c, l_i) + \delta(l_i, l_{i+1}) + \delta(l_{i+1}, c) \end{cases} & \text{c.c.} \end{cases}$$

¿Complejidad? Depende de cómo se calculen los $\delta(\cdot, \cdot)$. Los podemos precalcular en $O(n^3)$ y luego hom se calcula en $O(k)$.

Fin

¡Fin!